

FinVeritas: from code to a monitored service

How our Streamlit credit-analysis app is now tested, built, deployed, orchestrated and monitored, while staying simple enough to explain.

01 · CI/CD

GitHub Actions

test → build → deploy

02 · CONFIG

Ansible

packages, users, files

03 · CONTAINERS

Docker and Kubernetes

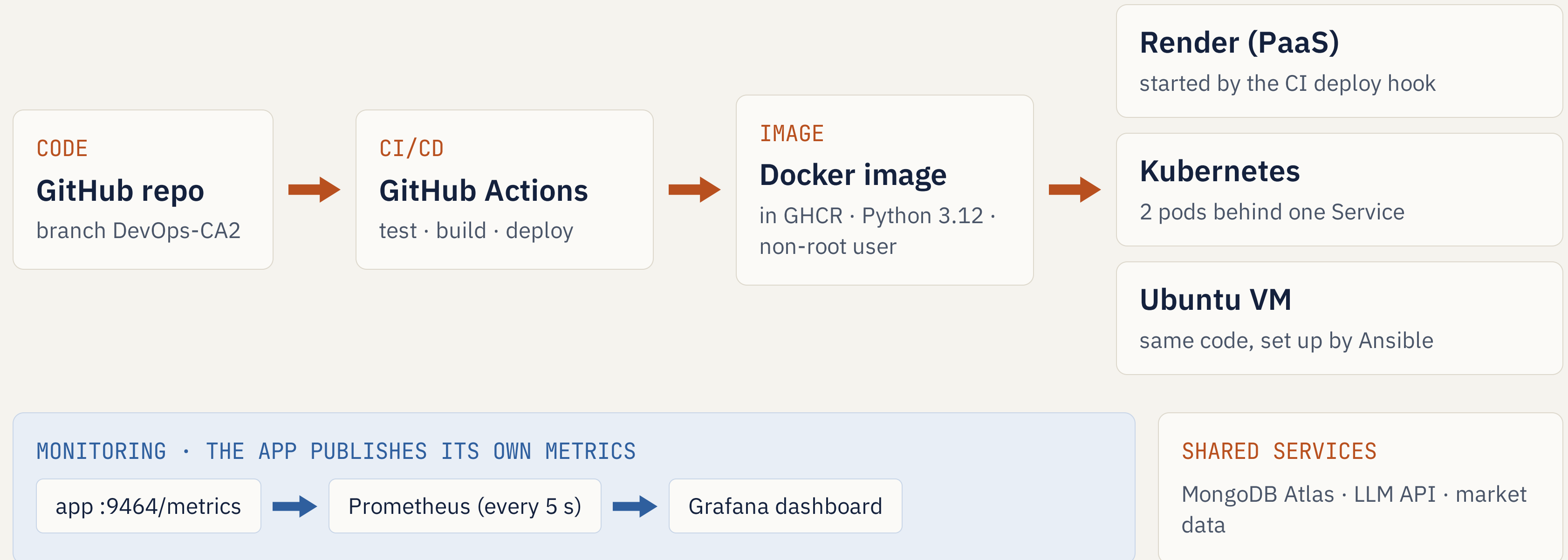
rolling update and rollback

04 · MONITORING

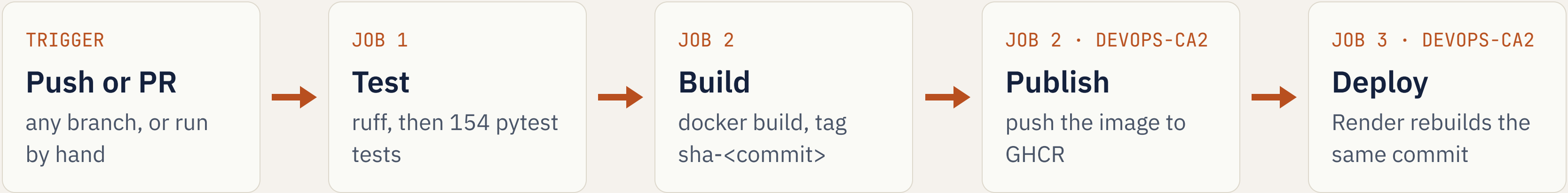
Prometheus and Grafana

uptime, latency, error rate

One codebase, three ways to run it



Every push tested, only DevOps-CA2 shipped



A failed job stops the run, so code that fails its tests is never deployed.

What runs when				
Event	Test	Build	Publish	Deploy
Branch or PR	yes	yes	no	no
Push to DevOps-CA2	yes	yes	yes	yes

After the deploy

Kubernetes: a new pod starts and becomes ready before an old one stops. **kubectl rollout undo** goes back the same way.

Monitoring: Prometheus reads /metrics every 5 s; Grafana shows uptime, p95 latency and error rate.

What went wrong, and how we fixed it

Pipeline too complex

Why: the first version had 7 jobs and ~200 lines we could not explain. **Fix:** rebuilt as 3 jobs: test, build, deploy.

Pods slow to stop

Why: the start command's shell swallowed the stop signal; old pods hung for 30 s. **Fix:** Python is the main process: 11 s to 1 s.

Keeping secrets out

Why: .env holds the database URI and JWT key. **Fix:** never in git or the image; Ansible copies it as 0600; Kubernetes uses a Secret.

Measuring Streamlit

Why: no request hooks, and `st.stop()` raises an exception. **Fix:** time each page run; only real errors count, and a test checks it.

Zero-downtime updates

Why: the local cluster cannot see local images, and new pods need time to start. **Fix:** kind load, a readiness probe, `maxUnavailable 0`.

Demo tooling surprises

Why: Chrome forced HTTPS for the host "app" and ran out of shared memory. **Fix:** use the container's full name; give Chrome 1 GB.

Six lessons we take forward

01

Simple beats complete

If we cannot explain a part, we do not need it yet. Add pieces when a real need appears.

03

Verify for real

A fresh Ubuntu server, a real cluster, a live dashboard. The demos found bugs that reading never would.

05

Health checks drive everything

A readiness probe gates every rollout, and the up metric is our uptime.

02

Automate the checks

Lint and 154 tests run on every push; a failed job stops the deploy.

04

Describe the end state

Ansible and Kubernetes say what should exist, so a re-run changes nothing (changed=0).

06

Least privilege by default

A non-root container, a service user nobody can log in as, and secrets never in git.

Next steps: add MongoDB and alert rules to the monitoring stack, and serve the app over HTTPS.